CS550 Final Project Proposal: N-Body Parallelization

Fady Zaky, Sakina Lord, Larry Griffith, Graceson Mulé, Brady Wisniewski

School of Informatics, Computing & Cyber Systems
Northern Arizona University, Flagstaff, AZ, U.S.A.

I. Introduction

N-body simulations attempt to predict how multiple objects of arbitrary mass and initial velocities behave when each is affected by the others' gravity. The motion of each body depends on the motion and gravity of all others in the system, which, in systems including more than two bodies, requires large computational effort to approximate the motion of the whole system. The standard N-body problem has a computational complexity of $O(N^2)$ due to the direct summation between every pair of bodies[1]. This limits the number of bodies that can be reasonably modeled and poses a challenge for researchers trying to approximate the motion of an entire asteroid field or discover new bodies based on that motion. Current sequential algorithms have improved upon this time complexity, but still require large amounts of compute power and time, especially in simulations involving more than 10^6 bodies[1].

Current parallel algorithms improve upon the time required, but the results are less precise when bodies are far from each other's gravitational fields[2]. One such algorithm is Barnes-Hut, which is a spatial tree-based method. It uses a hierarchical oct-tree to group particles based on location, then selects the movement modeling algorithm to use based on the distance between a given body and all others in the system; bodies close enough together are treated as a single mass[2]. By approximating clusters as single bodies, Barnes-Hut reduces the number of pairwise calculations required at the expense of precision, lowering the computational cost of simulating N bodies from $O(N^2)$ to $O(NlogN)$.

II. Preliminary Solution

Our project will implement a matrix-based solution based on tiled all-pairs and will be informed by and compared to both the NBODY6++GPU method and the OpenMP-forward PETAR algorithm discussed in Wang et al.[2]. Tiled all-pairs is a parallel implementation of direct gravitational N-body summation that divides bodies into fixed-size tiles. Threads sum the bodies within each tile, then continue to the remaining tiles in the system.

The force of a body i is given by the sum of the pairwise interactions with all other particles j[1][2]:

$$a_i = -G \sum_{j \neq i}^{n} m_j \frac{r_i - r_j}{|r_i - r_j|^3}$$

Since this method sums exact pairwise forces, it will have better precision than the Barnes-Hut tree model while maintaining a level of speedup attributed to parallel solutions. It will use a maximum of 10^4 bodies, given that we are more limited in time and resources than the current literature, and will be compared to a sequential tiled all-pairs approach using benchmarking techniques discussed in class.

This is supported by legacy literature published by NVIDIA showcasing their implementation of tiling for fast N-body simulation in CUDA[3]. This method gives each CUDA thread block of p threads p bodies to simultaneously compute accelerations for and uses barrier synchronization to ensure all threads have finished loading data before calculations begin and before moving to the next tile[3]. By using a shared memory source that holds only one tile at a time, NVIDIA's implementation was able to alleviate a memory bottleneck which had historically slowed the problem[3].

Our implementation will use this idea of reducing the total bodies into smaller tiles to improve cache locality. Thus, each tile size will need to be chosen such that they match our machines' L1 cache capacity. We will also parallelize the summations for each target body using OpenMP's `#pragma omp parallel for` and write only to the global array once at the end of all iterations within a tile.

III. Challenges

Even though the most recent algorithms have made significant progress on reducing the computational complexity and overall time of the N-body problem, there remain challenges to address. In our implementation, we anticipate unpredictable locations of imbalanced loads based on where clusters of bodies form, as well as memory bottlenecks similar to those encountered in NBODY6++GPU[2]. Our implementation will also not be a one-to-one comparison with those in the literature due to the decreased number of bodies ($N = 10^4$ versus $N = 10^6$) since we do not have the same amount of computational power at our disposal.

IV. Preliminary Benchmarks

In the literature, PETAR simulated close-range bodies up to 12 times faster than the legacy NBODY6++GPU algorithm. We do not expect to see this scale of speedup in our own methods due to using CPU cores rather than more matrix-specialized GPUs and because the tiled all-pairs approach does not use the approximations that reduce the computational complexity of the Barnes-Hut method. We will compare our approach to the sequential algorithm as a baseline for speedup and to the Barnes-Hut method for both an evaluation of speedup and accuracy.

V. Milestones

Milestones have been agreed upon by the group and are outlined in Table I. Fady will add tasks in the form of Issues to the shared GitHub repository to outline specific tasks to be implemented by team members.

TABLE I: Outline of Milestones

Date	Milestone	Description
11/5	Project Proposal	Introducing the project, as well as defining expected challenges.
11/5	Creation of shared repo	On GitHub under Fady.
11/10	Sequential Pseudocode	Development of pseudocode for the sequential implementation.
11/11	Sequential Implementation	Based on tiled all-pairs literature, used as a benchmark.
11/12	Parallelized Pseudocode	Development of pseudocode for the parallelized implementation.
11/16	First Phase Implementation of Parallelized Code	Implementation and debugging.
11/19	Preliminary Report	Overview of the project and its current state.
11/29	Second Phase Implementation of Parallelized Code	Implementation and debugging.
12/02	Benchmark Measurements	Taking final benchmarks to see how parallel code performs compared to sequential code.
12/07	Draft Final Report	Hosted in the team repository.
12/08	Team Reviews of Final Report	Reviews by the team of the final report completed.
12/10	Final Report Submission	Overall report on the project, including approach, results, and discussion.

References

[1] W. Dehnen and J. I. Read, "N-body simulations of gravitational dynamics," *The European Physical Journal Plus*, 2011.

[2] M. N. K. Wang, Long; Iwasawa and J. Makino, "Petar: a high-performance n-body code for modelling massive collisional stellar systems," *Monthly Notices of the Royal Astronomical Society*, pp. 536–555, 2020.

[3] M. Nyland, Lars; Harris and J. Prins, *GPU Gems 3: Fast N-Body Simulation with CUDA*. Nvidia Corporation, 2007, ch. 31. [Online]. Available: https://github.com/NVIDIA/warp/blob/main/warp/examples/tile/example˙tile˙nbody.py